If the passed hash contains any keys not defined as parameters, Ruby raises a runtime error unless the method also declares a double splat argument. In that case, the double splat receives the excess key-value pairs from the passed hash. If the passed hash is missing keys that are defined as required parameters without default values, Ruby raises a runtime error.

```
def keywords1(a, b: 2, c: 3)
  "a = #{a}, b = #{b}, c = #{c}"
end

keywords1(99, d: 22, e: 33)
```

produces:

```
        from prog.rb:5:in `<main>'
prog.rb:1:in `keywords1': unknown keywords: :d, :e (ArgumentError)
```

```
def keywords2(a, b: 2, c: 3, **rest)
  "a = #{a}, b = #{b}, c = #{c}, rest = #{rest}"
end

keywords2(99, d: 22, e: 33)    # => a = 99, b = 2, c = 3, rest = {:d=>22, :e=>33}
```

As with hash literals, if there is an existing local value with the same name as the keyword argument, you can pass the argument without a value and the local value will be found and used:

```
x = 10
y = 5
foo(x:, y:)
```

A block may be associated with a method call using either a literal block (which must start on the same source line as the last line of the method call) or an argument containing a reference to a lambda, Proc, or Method object prefixed with an ampersand character. If the object prefixed by a block responds to to_proc, then to_proc is invoked and the resulting proc passed to the the method.

```
def some_method
  yield
end

some_method { }
some_method do
end

a_proc = lambda { 99 }
some_method(&a_proc)
```

Ruby sets the value of block_given? to reflect the availability of a block associated with the call, regardless of the presence of a block parameter. An explicit block parameter will be set to nil if no block is specified on the call to a method.

```
def other_method(&block)
  puts "block_given = #{block_given?}, block = #{block.inspect}"
end
other_method { }
other_method
```

produces:

```
block_given = true, block = #<Proc:0x00000001021a8c38 prog.rb:4>
```